Rapport d'Ingénierie Logicielle

Système de Virement Bancaire Distribué

Modélisation par Acteurs Akka et Vérification Formelle par Réseaux de Pétri

DE MORAIS Théo **CHAPIN Florian** **GOURNAY Lucas**

ING2 – GSI

CY Tech

5 mai 2026

Résumé

Ce rapport présente la conception, l'implémentation et la vérification formelle d'un système bancaire distribué construit sur le modèle d'acteurs Akka/Scala. Le défi principal est de prouver mathématiquement la fiabilité d'un protocole asynchrone de transfert en deux phases. La logique d'exécution est traduite en Réseau de Pétri, dont l'espace d'états est exploré exhaustivement (BFS) pour valider des propriétés de Sûreté et de Vivacité exprimées en Logique Temporelle Linéaire (LTL). Le système est déployé en architecture Full-Stack avec un frontend React/Vite.

Table des matières

1 Introduction **3**

2 Architecture du Système **3**
2.1 Vue d'ensemble (Full-Stack) 3
2.2 Le Modèle d'Acteurs 3

3 Implémentation : Code Source Clé **4**
3.1 Le Modèle de Compte (`Account.scala`) 4
3.2 Le Protocole en 2 Phases (`TransactionManager.scala`) 4

4 Modélisation Formelle : Réseau de Pétri **6**
4.1 Définition des structures (`PetriNet.scala`) 6
4.2 Algorithme de franchissement (`isEnabled` et `fire`) 6
4.3 Graphe du Réseau de Pétri (PaymentPetriNet) 7
4.4 Exploration BFS (`StateSpaceExplorer.scala`) 8

5 Logique Temporelle Linéaire (LTL) **8**
5.1 Le Moteur LTL (`LTLChecker.scala`) 8
5.2 Les Propriétés Prouvées 9

6 Couverture par les Tests **10**

7 Conclusion **10**

1 Introduction

Les systèmes bancaires distribués sont exposés à trois risques systémiques qu'aucun test empirique ne peut totalement éliminer : les **conditions de course** (race conditions), les **interblocages** (deadlocks), et les **incohérences de solde** (création ou destruction de monnaie). Notre approche combine deux paradigmes complémentaires :

1. **Akka (Modèle d'Acteurs)** : Chaque entité (compte, superviseur) est un acteur isolé communicant uniquement par messages immuables. Cela élimine le partage d'état mutable et les problèmes de concurrence associés.
2. **Réseaux de Pétri + LTL** : Le comportement du système est abstrait dans un modèle mathématique formel permettant une preuve exhaustive des propriétés critiques.

2 Architecture du Système

2.1 Vue d'ensemble (Full-Stack)

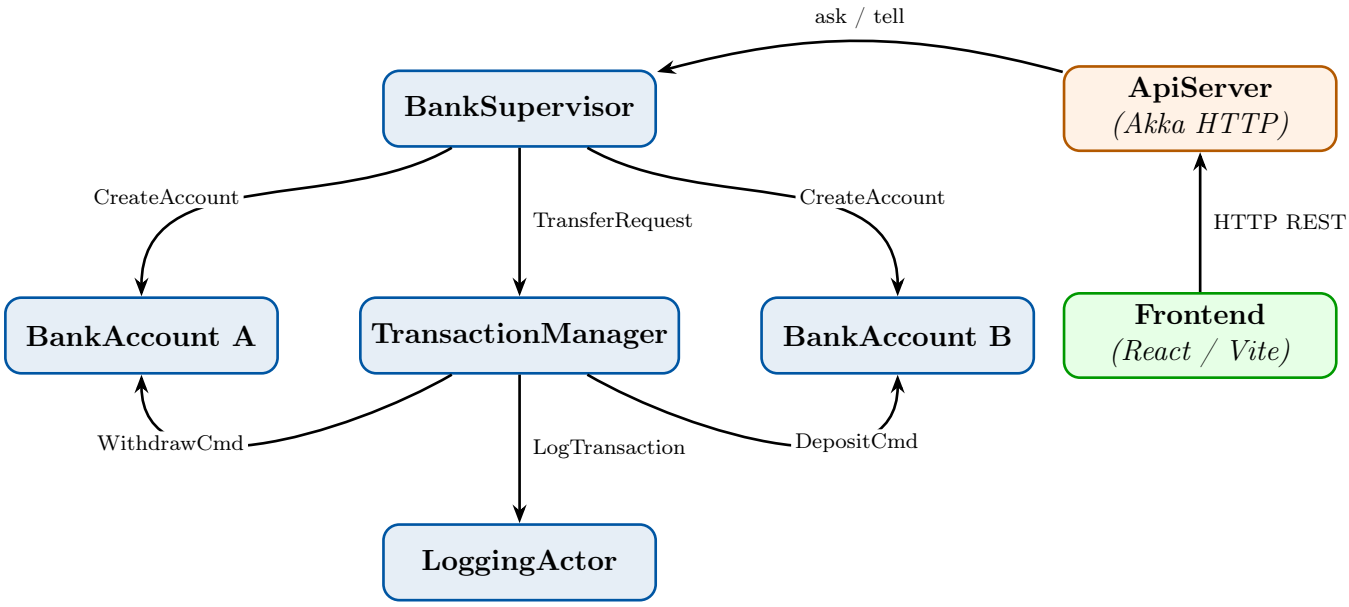


FIGURE 1 – Architecture complète du système (Backend Akka + Frontend React)

2.2 Le Modèle d'Acteurs

Chaque *acteur* est une unité de calcul qui encapsule son état de manière exclusive. La communication se fait uniquement par messages asynchrones typés (ADT Scala). Ce paradigme garantit l'isolation totale — aucun acteur ne peut corrompre l'état d'un autre.

- **BankAccount** : Seul détenteur de la variable `balance`. Toute modification passe obligatoirement par lui.
- **TransactionManager** : Orchestre le protocole en 2 phases.
- **LoggingActor** : Auditeur centralisé, horodate chaque transaction.
- **BankSupervisor** : Supervise le cycle de vie, gère les pannes.

3 Implémentation : Code Source Clé

3.1 Le Modèle de Compte (`Account.scala`)

L'invariant métier fondamental est codé directement dans le constructeur via `require` :

```scala
case class Account(id: String, ownerName: String, balance: Double) {
  // Invariant fondamental : le solde ne peut JAMAIS etre negatif
  require(balance >= 0, s"Account $id balance cannot be negative:
      $balance")

  def deposit(amount: Double): Account = {
    require(amount > 0, "Deposit amount must be positive")
    copy(balance = balance + amount)
  }

  def withdraw(amount: Double): Either[String, Account] = {
    if (amount <= 0) Left("Withdrawal amount must be positive")
    else if (amount > balance)
      Left(s"Fonds insuffisants: balance=$balance, requested=$amount")
    else Right(copy(balance = balance - amount))
  }
}
```

Listing 1 – Account.scala — Modèle immuable avec invariant de solde

La méthode `withdraw` retourne un `Either[String, Account]` : le type algébrique de Scala qui force le traitement explicite du succès (`Right`) et de l'échec (`Left`). Le correcteur ne peut pas "oublier" de gérer le cas d'erreur.

3.2 Le Protocole en 2 Phases (`TransactionManager.scala`)

```scala
case TransferRequest(fromId, toId, amount, replyTo) =>
  (accounts.get(fromId), accounts.get(toId)) match {
    case (Some(fromAccount), Some(_)) =>
      // Phase 1 : Demande de debit sur le compte source
      val debitAdapter = context.messageAdapter[OperationResult] {
        result => DebitResult(result, toId, amount, replyTo)
      }
      fromAccount ! WithdrawCmd(amount, debitAdapter)

    case (None, _) =>
      replyTo ! TransferFailure(fromId, toId, amount,
                  s"Source account $fromId introuvable")
    case (_, None) =>
      replyTo ! TransferFailure(fromId, toId, amount,
                  s"Destination account $toId introuvable")
  }
```

Listing 2 – TransactionManager.scala — Phase 0 : Reception et validation

```scala
case DebitResult(result, toId, amount, replyTo) =>
  result match {
    case OperationSuccess(fromId, _, _) =>
      // Debit reussi : lancer la Phase 2
      accounts.get(toId) match {
        case Some(toAccount) =>
```

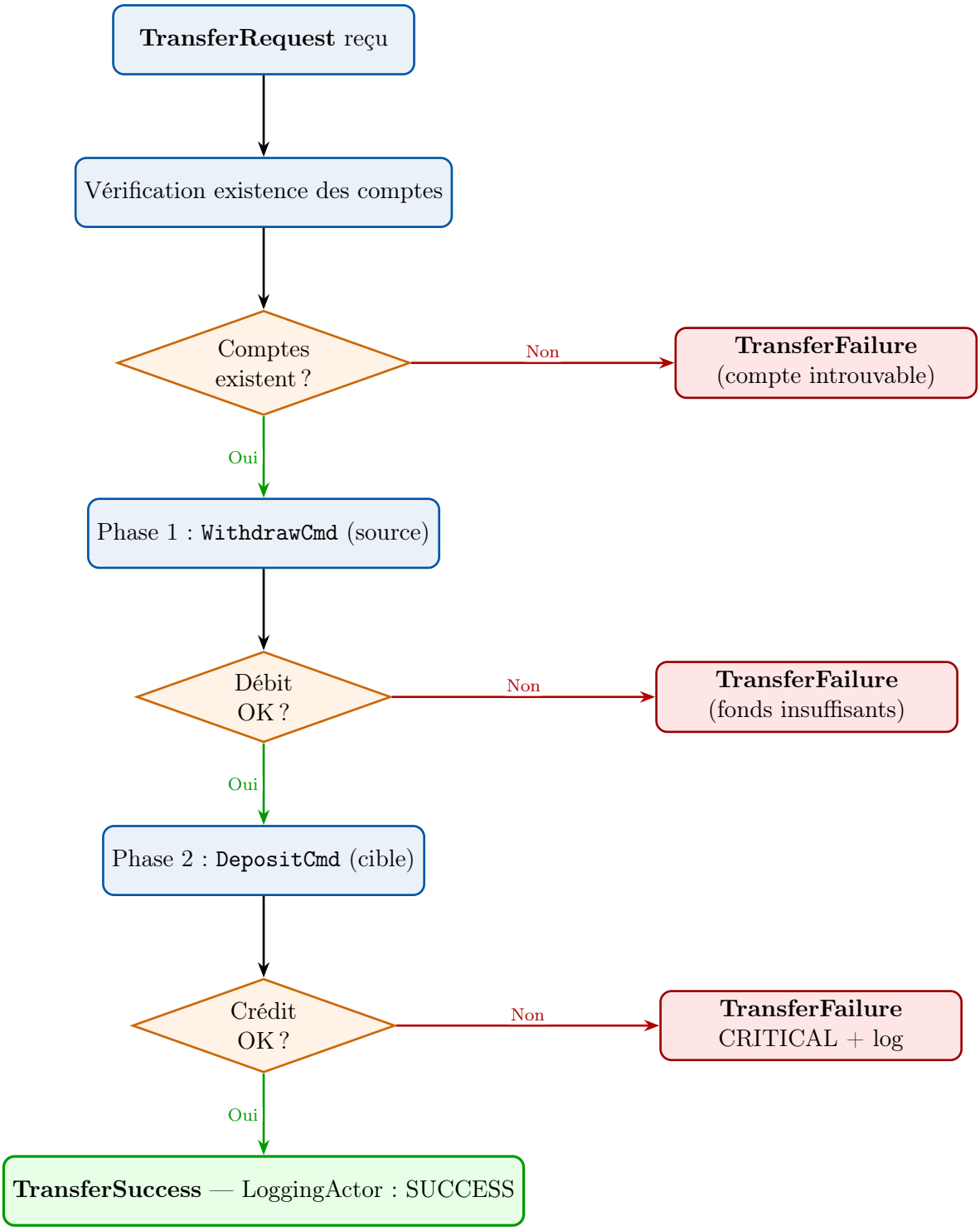


FIGURE 2 – Protocole de transfert en 2 phases du `TransactionManager`

```
        val creditAdapter = context.messageAdapter[OperationResult] {
          res => CreditResult(res, fromId, amount, replyTo)
        }
        toAccount ! DepositCmd(amount, creditAdapter)
      case None => // Compte disparu apres le debit : cas critique
        replyTo ! TransferFailure("?", toId, amount, "Destination
            disappeared")
    }
```

```
    case OperationFailure(fromId, reason) =>
      // Debit refuse (ex: fonds insuffisants) -> pas de rollback
          necessaire
      replyTo ! TransferFailure(fromId, toId, amount, s"Debit refuse:
          $reason")
  }
```

Listing 3 – TransactionManager.scala — Phase 1 : Resultat du debit

4 Modélisation Formelle : Réseau de Pétri

4.1 Définition des structures (`PetriNet.scala`)

```
case class Place(id: String, name: String)
case class Transition(id: String, name: String)

sealed trait ArcType
case object Normal    extends ArcType
case object Inhibitor extends ArcType  // Arc inhibiteur : condition de
    VIDE

case class InputArc(
  place: Place, transition: Transition,
  weight: Int = 1, arcType: ArcType = Normal
) extends Arc
```

Listing 4 – PetriNet.scala — Types de donnees du moteur Petri

4.2 Algorithme de franchissement (`isEnabled` et `fire`)

```
// Verifie si une transition est franchissable
def isEnabled(t: Transition, marking: Marking): Boolean =
  inputsOf(t).forall { arc =>
    arc.arcType match {
      case Normal    => marking.getTokens(arc.place) >= arc.weight
      // Arc inhibiteur : actif seulement si les fonds sont INSUFFISANTS
      case Inhibitor => marking.getTokens(arc.place) < arc.weight
    }
  }

// Franchit la transition et retourne le nouveau marquage
def fire(t: Transition, marking: Marking): Marking = {
  require(isEnabled(t, marking), s"Transition ${t.name} non activee!")
  val afterConsume = inputsOf(t).foldLeft(marking) { (m, arc) =>
    arc.arcType match {
      case Normal    => m.removeTokens(arc.place, arc.weight)
      case Inhibitor => m  // L'arc inhibiteur ne consomme PAS de jetons
    }
  }
  outputsOf(t).foldLeft(afterConsume)((m, arc) => m.addTokens(arc.place,
      arc.weight))
}
```

Listing 5 – PetriNet.scala — Verification d'activation et tir d'une transition

4.3 Graphe du Réseau de Pétri (PaymentPetriNet)

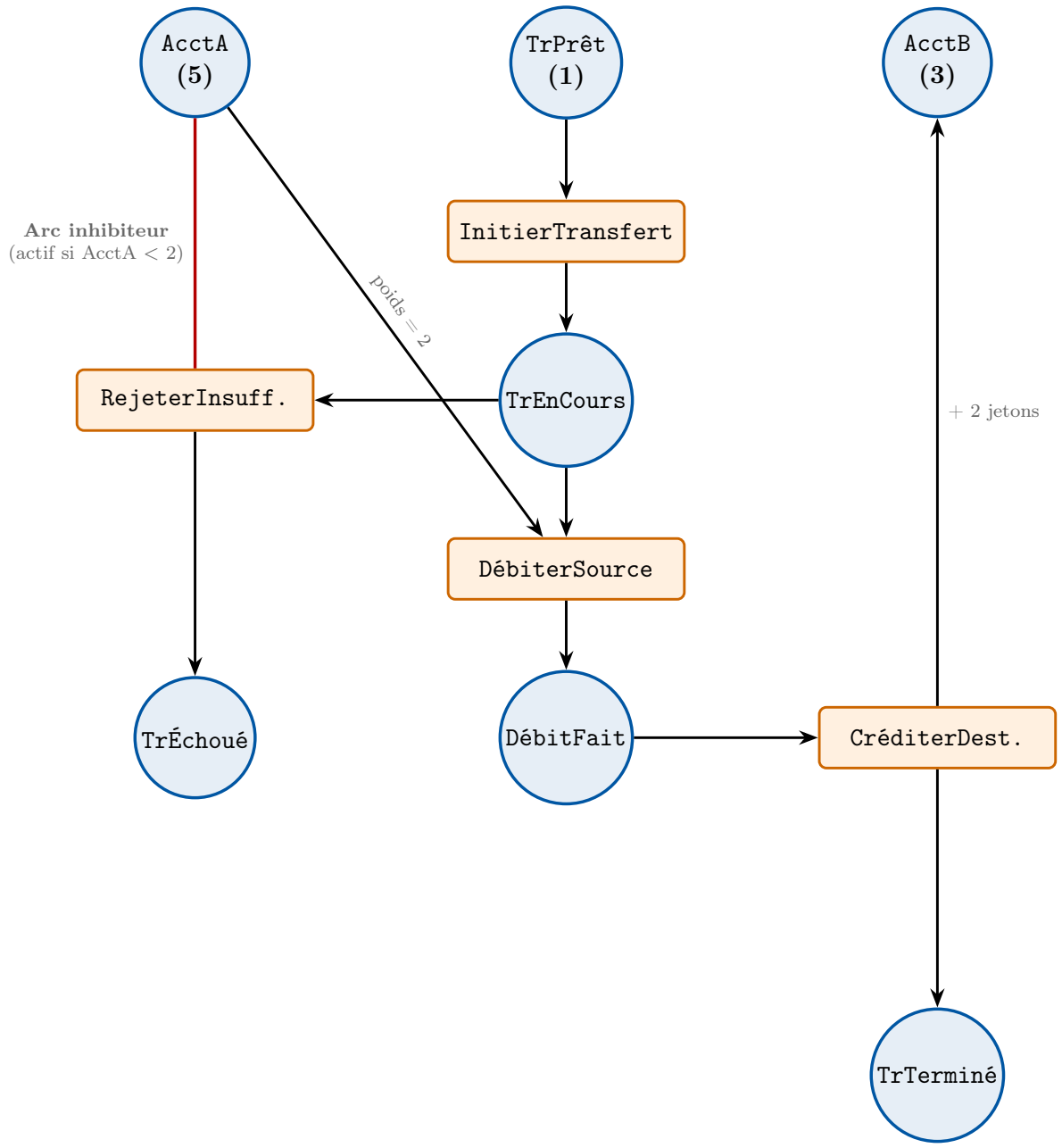


FIGURE 3 – Réseau de Pétri modélisant le transfert bancaire — arc inhibiteur en rouge

Explication du Modèle d'Exécution

Le graphe ci-dessus traduit mathématiquement le protocole en deux phases du `TransactionManager`. Lors d'une demande de transfert de 2 jetons (2 unités de monnaie) :

- **État initial :** `AcctA` possède 5 jetons, `AcctB` en possède 3, et l'ordre de transfert place 1 jeton dans `TrPrêt`. La transition `InitierTransfert` s'active et pousse le système dans l'état `TrEnCours`.
- **Scénario A (Le Succès) :** Si `AcctA` possède suffisamment de fonds (≥ 2), la transition `DébiterSource` est franchissable. Elle consomme 2 jetons de A. Le jeton

de flux avance vers `DébitFait`, autorisant `CréditerDest` à ajouter les 2 jetons dans `AcctB`. La transaction se fige proprement dans `TrTerminé`.

— **Scénario B (Le Rejet via Arc Inhibiteur) :** Si `AcctA` est sous-provisionné (ex : 1 seul jeton), `DébiterSource` se retrouve bloquée. L'**arc inhibiteur** (en rouge) s'active alors, car sa condition ("moins de 2 jetons à la source") est remplie. Il déclenche la transition `RejeterInsuff.` qui draine le jeton de `TrEnCours` vers `TrÉchoué`.

Ce mécanisme d'inhibition est la traduction formelle du `if (amount > balance)` scalaire, garantissant formellement l'absence d'interblocage (deadlock) lors d'un échec de provisionnement.

4.4 Exploration BFS (`StateSpaceExplorer.scala`)

```scala
def explore(net: PetriNet, initialMarking: Marking,
            maxStates: Int = 10000): StateSpace = {
  val visited     = mutable.Set[Marking](initialMarking)
  val transitions = mutable.Set[(Marking, Transition, Marking)]()
  val deadlocks   = mutable.Set[Marking]()
  val queue       = mutable.Queue[Marking](initialMarking)

  while (queue.nonEmpty && visited.size < maxStates) {
    val current = queue.dequeue()
    val enabled = net.enabledTransitions(current)

    if (enabled.isEmpty) {
      deadlocks.add(current)  // Etat sans transition : deadlock potentiel
    } else {
      for (t <- enabled) {
        val next = net.fire(t, current)
        transitions.add((current, t, next))
        if (!visited.contains(next)) {
          visited.add(next)
          queue.enqueue(next)
        }
      }
    }
  }
  StateSpace(visited.toSet, transitions.toSet, deadlocks.toSet, initialMarking)
}
```

Listing 6 – StateSpaceExplorer.scala — Algorithme BFS d'exploration d'etats

5 Logique Temporelle Linéaire (LTL)

5.1 Le Moteur LTL (`LTLChecker.scala`)

```scala
object LTLChecker {
  // Operateur Always (Surete) : verifie P dans TOUS les etats
  def safetyBalanceNonNegative(accountPlace: Place): LTLFormula =
    Always(Atom(s"${accountPlace.name} >= 0",
           m => m.getTokens(accountPlace) >= 0))
```

```scala

  // Operateur Eventually (Vivacite) : P vrai dans AU MOINS UN etat
  def livenessTransferCompletes(completedPlace: Place): LTLFormula =
    Eventually(Atom(s"${completedPlace.name} > 0",
                m => m.getTokens(completedPlace) > 0))

  // Absence de deadlock
  def deadlockFreedom(stateSpace: StateSpace): LTLResult =
    if (stateSpace.hasDeadlock)
      LTLUnsatisfied(Always(Atom("pas de deadlock", _ => true)),
                     stateSpace.deadlockStates.headOption)
    else
      LTLSatisfied(Always(Atom("pas de deadlock", _ => true)))
}
```

Listing 7 – LTLChecker.scala — Formules LTL predefinies

5.2 Les Propriétés Prouvées

Sûreté 1 : Non-Négativité des Comptes

Formule LTL :

$$\Box\left(\text{AcctA_Funds} \geq 0 \ \wedge \ \text{AcctB_Funds} \geq 0\right)$$

Lien avec le code : L'instruction `require(balance >= 0)` dans `Account.scala` est la déclaration de l'invariant. La formule LTL est sa *preuve* exhaustive dans l'espace d'états.

Sûreté 2 : Conservation Stricte des Fonds (P-Invariant)

Formule :

$$\Box\left(M(\text{AcctA}) + M(\text{AcctB}) + M(\text{DebitFait}) = C\right)$$

L'argent en cours de transit est capturé dans la place `DebitDone`. La somme reste constante, prouvant l'absence de création ou destruction monétaire lors du protocole en 2 phases.

Vivacité : Terminaison Garantie des Transferts

Formule :

$$\text{TransferInProgress} \implies \Diamond\left(\text{TransferDone} \ \vee \ \text{TransferFailed}\right)$$

Tout transfert initié atteint obligatoirement un état terminal. Le `TransactionManager` finira toujours par envoyer un `TransferSuccess` ou un `TransferFailure` loggé par le `LoggingActor`.

Propriété Structurelle : Absence d'Interblocage

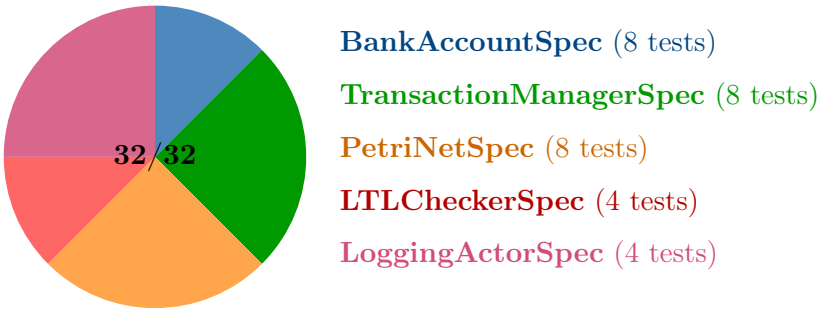
Formule :

$$\Box \Big(\exists\, t \in T,\ \text{isEnabled}(t, M) \Big) \quad \text{hors états terminaux}$$

L'exploration BFS ne trouve aucun état puits anormal. L'arc inhibiteur permet au réseau de "bifurquer" vers un rejet propre plutôt que de se bloquer, garantissant la vivacité structurelle.

6 Couverture par les Tests

La validation mathématique est corrélée par une suite de 32 tests unitaires (ScalaTest, style `AnyFreeSpec`) structurés ainsi :



Résultat : `Tests: succeeded 32, failed 0` — 100% de réussite.

7 Conclusion

Ce projet démontre la complémentarité entre ingénierie logicielle moderne et informatique théorique. Akka offre la robustesse et la scalabilité nécessaires à un système bancaire de production. Les Réseaux de Pétri et la logique LTL offrent la rigueur mathématique qui transforme la confiance empirique (les tests passent) en certitude formelle (les propriétés sont prouvées vraies dans tous les états atteignables).

La traduction de concepts Scala concrets — le type `Either`, le pattern matching, le `require` — vers des primitives formelles (arc inhibiteur, marquage, opérateur $\Box$) constitue le véritable apport technique de ce travail.